SZYBKIE TWORZENIE REPOZYTORIUM:
echo "# Nazwa Projektu" > README.md
git init
git add .
git commit -m "Initial commit"
git remote add origin <URL_REPOZYTORIUM>
git branch -M main
git push -u origin main

Skróty w Git Bush:
Ctrl + c = Ctrl + Ins
Ctrl + v = Shift + Ins
Ctrl + c = przerwanie działania (jak się np. zawiesi)

JEŚLI W NAZWIE PLIKU/FOLDERU WYSTĘPYJE SPACJA NALEŻY W KONSOLI PRZED SPACJĄ DAĆ: \

`clear` – Czyści konsole.

`cd /d` – Przejście na dysk D.

`cd ścieżkaDoFolderu` – Przejście do wybranego folderu.

`cd ..` – Powrót do poprzedniego (nadrzędnego) folderu.

`mkdir nazwaFolderu` – stworzenie nowego folderu.

ls – lista plików i folderów w aktualnie otwartym folderze.

`mv staraNazwa nowaNazwa` – Zmienia nazwę pliku lub folderu.

`mv nazwaPlikuLubFolderu ścieżkaDoNowegoFolderu` – Przeniesienie pliku / folderu.

Aby przenieść plik/folder do folderu nadrzędnego należy:

`cd lokalizacjaPliku`- przejść do lokalizacji przenoszonego pliku/folderu

`mv nazwaPlikuLubFolderu ..` – Przeniesienie pliku/folderu do folderu nadrzędnego.

`cp nazwaPlikuLubJegoLokalizacja lokalizacjaDocelowa` – Kopiowanie pliku.

`cp –r nazwaFolderuLubJegoLokalizacja lokalizacjaDocelowa` – Kopiowanie folderu (-r działanie na folderach).

`rm nazwaPliku` – Usuwa plik.

`rm –r nazwaFolderu` – Usuwa pusty folder.

`rm –rf nazwaFolderu` – Usuwa folder z zawartością (-f daje „uprawnienie administratora”).

`rm –rf *` - Usuwa wszystko z aktualnie otwartego folderu.

`touch plik.rozszerzenie` – Utworzenie pustego pliku w aktualnym folderze.

`echo „Jakiś tekst” > plik.rozszerzenie` – Utworzenie nowego pliku jeśli nie istniał i zapisanie w nim: Jakiś tekst. Jeśli plik istniał to jego zawartość zostanie zastąpiona przez: Jakiś tekst.

`code . plik.rozszerzenie` – Tworzy nowy plik jeśli nie istniał i otwiera na nim **domyślny** edytor kodu. Jeśli plik istniał pozwala na jego edycje.

`nano plik.rozszerzenie` – To samo co powyżej tylko otwiera edytor tekstu **Nano** (^ = Ctrl).

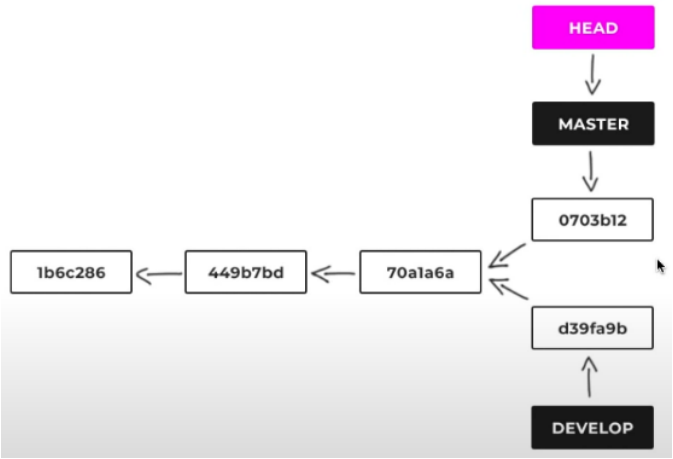
`vim plik.rozszerzenie` – To samo co powyżej tylko edytor tekstu **Vim** (Vim jest trudniejszy od Nano – trzeba znać skróty klawiszowe).

Skróty/komendy	***Opis***	***Tryb***
h	przesunięcie w lewo	normal
l	przesunięcie w prawo	normal
j	przesunięcie w dół	normal
k	przesunięcie w górę	normal
i	przejście w tryb **insert** (pisanie, wprowadzanie tekstu)	normal
Ctrl+d	przesunięcie ekranu o połowę jego wysokości w dół	normal
Ctrl+u	przesunięcie ekranu o połowę jego wysokości w górę	normal
Ctrl+f	przesunięcie ekranu o całą jego wysokość w dół	normal
Ctrl+b	przesunięcie ekranu o całą jego wysokość w górę	normal

gg	przejście do pierwszej linii w pliku	normal
G	przejście do ostatniej linii w pliku	normal
H	przejście do pierwszej *widocznej* na ekranie linii (skrót od High)	normal
M	przejście do środkowej *widocznej* na ekranie linii (skrót od Medium)	normal
L	przejście do ostatniej *widocznej* na ekranie linii (skrót od Low)	normal
Esc	wyjście z trybu **insert** lub **replace** do **normal**	insert
a	przejście do trybu **insert** wraz z przesunięciem w prawo	normal
o	dodanie nowej linii poniżej bieżącej i przejście do trybu **insert** na początku nowej linii	normal
I	przejście do trybu **insert** wraz z przesunięciem do pierwszego znaku drukowalnego w linii	normal
A	przejście do trybu **insert** wraz z przesunięciem na koniec linii	normal
O	dodanie nowej linii powyżej bieżącej i przejście do trybu **insert** w nowej linii	normal
:	przejście w tryb **command-line**	normal
<NR>	przejście do linii o podanym numerze	command-line
q	wyjście z edycji pliku	command-line
w	zapisanie pliku	command-line
w <NAZWA_PLIKU>	zapisanie pliku pod nazwą <NAZWA_PLIKU>	command-line
wq	zapisanie pliku i wyjście z edycji	command-line
e	ponowne otwarcie bieżącego pliku	command-line

e <NAZWA_PLIKU>	otwarcie w bieżącym oknie pliku <NAZWA_PLIKU> do edycji	command-line
q!	wyjście z edycji pliku, nawet jeśli zmiany nie zostały zapisane	command-line
qa	wyjście z edycji wszystkich otwartych plików (quitall)	command-line
!<SYSCMD>	uruchomienie dowolnej komendy powłoki systemu.	command-line
.	powtórzenie ostatnio wykonanej akcji	normal
u	wycofanie ostatniej akcji (undo, Ctrl + z)	normal
Ctrl+r	ponowne wykonanie wycofanej akcji	normal

Repozytorium lokalne (z ang. Local repository) – Repozytorium zapisane na dysku komputera na którym pracujemy.
Repozytorium zdalne (z ang. Remote repository) – Repozytorium które jest połączone z lokalnym, ale znajduje się poza naszym komputerem (najczęściej na serwerze lub serwisach github, gitlab itp.).
Origin – Alias / nazwa pod którą będzie przechowywany adres / link do głównego zdalnego repozytorium. Można wybrać dowolną nazwę lecz ta jest ogólnie stosowana.
Commit – Powoduje zapisanie zmian poprzez stworzenie nowego Snapshota. Każdy snapshot ma unikalne Id oraz komentarz. Commit – (z ang. Popełnić).
HEAD – Wskaźnik HEAD decyduje która wersja projektu (czyli jaki Snapshot) jest aktualnie wczytana w folderze roboczym.
Branch – to wskaźnika wskazujący na konkretny Snapshot (Branch wskazuje na Snapshot a HEAD na Branch). Nie tworzy nowego zapisu i nie marnuje pamięci. O jego wyborze decyduje wskaźnik Head. Branch jest to odgałęzienie od głównej linii projektu. Tworzy się je aby oddzielić pracę nad nowymi funkcjami projektu od jego głównej już działającej wersji.



Wprowadzenie danych użytkownika git (nie muszą być takie same jak np. na GitHub)

`git config --global user.email "you@example.com"` – Przypisanie email.

`git config --global user.name "Your Name"` – Przypisanie nazwy.

`git config --global credential.helper store` – Zapamiętuje wprowadzone dane.

Aby wyświetlić aktualnie przypisany email lub nazwę użyj: git config --global user.email lub git config --global user.name

`git config --global --list` – Wyświetlenie zapisanych nazwy i emaila.

`git config --global --unset-all user.name` – Usuwa wszystkie zapisane nazwy użytkownika.

`git config --list` – Wyświetla wszystkie informacje z Git Bash.

`git init` – Utworzenie git (systemu kontroli wersji) w aktualnym folderze.
Aby usunąć repozytorium usuń ukryty folder .git
`code .` – Włączenie domyślnego edytora kodu w aktualnym folderze.

WAŻNE w folderze roboczym można stworzyć plik .gitignore w którym wpisujemy nazwy plików lub folderów których git ma nie zapisywać (nie widzieć – ignorować).

`git status` – Sprawdzenie statusu repozytorium - zielone pliki dodane czerwone nie dodane do śledzenia.
`git add .` lub `git add "nazwa"` – Dodanie plików lub zmian w nich do systemu kontroli wersji (plik zielony).
`git reset "nazwa"` – Odwrotność add - usunięcie plików z systemu kontroli wersji (plik czerwony).

WAŻNE gdy zmiany nie zostały jeszcze zapisane (commit) i (plik świeci na czerwono) można **przywrócić** plik z poprzedniego zapisu (commita) komendą: `git restore nazwaPliku`

`git commit –m "Komentarz"` – Zapisanie dodanych plików / zmian z komentarzem – jakie zmiany wprowadzono. **Zalecana jest opcja po niżej ponieważ można oddzielać linie enterem.**
`git commit` – Wtedy otworzy się edytor tekstu Vim w którym trzeba wpisać komentarz commita (zapisu).

WAŻNE w edytorze Vim:
Aby wprowadzać tekst należy wcisnąć klawisz „i" .
Aby wyjść z terminalu należy wcisnąć Esc, użyć znaku „:" który ustawi kursor w wierszu poleceń i wpisać „wq" lub „wq!" (Jeśli wystąpi błąd readonly), a następnie wcisnąć Enter.
(w- zapis, q – wyjście, ! – siła, uprawniania administratora)

KOMENTARZ powinien zawierać: tytuł (max 50 znaków, w trybie rozkazującym) oddzielony pustym wierszem (Enterem) od opisu (max 72 znaki w 1 wierszu).

`git commit --amend -m "Nowy komentarz"` – Zmiana komentarza w ostatnio stworzonym commicie.

`git reset idCommitu` – Usuwa wszystkie commity utworzone po tym o wybranym id. Nie usuwa plików, można dodać je do kolejki: `git add .` i stworzyć nowy commit aby je zapisać: `git commit`.
`git reset --hard idCommitu` – Usuwa wszystkie commity wraz z plikami utworzone po tym o wybranym id. Przywraca do stanu z wybranego commita.

Przywracanie commotów i plików po usunięciu:
`git reflog` – Historia położenia wskaźnika HEAD (czytać od dołu do góry), należy skopiować id commitu do którego chcemy powrócić.
`git reset --hard idCommitu` – Przywraca pliki i commity do stanu commita o podanym id.

PODGLĄD HISTORII ZAPISÓW (COMMITÓW):
`git log` – Sprawdzenie historii zapisów (commitów), **wyjście z historii - wcisnąć klawisz „q"**
Flagi (parametry) git log (flagi można łączyć):
`git log --help` – Wszystkie flagi.
`git log --oneline` – Uproszczona lista utworzonych zapisów (commitów) wraz z ich Id.
`git log --oneline --author="Art"` – Uproszczona lista commitów utworzonych przez autora ze znakami: „Art" w nazwie autora. Wyświetlą się commity utworzone np. przez **Art**ura, B**art**ka.
`git log --grep="in"` – Wyszukanie po znakach występujących w komentarzu commita. Zostanie wyświetlony np. „**In**itiall commit".
`git log -2` – Wyświetli tylko 2 najnowsze commity.
`git log --oneline -- css/szablony.css` – Wyświetli uproszczoną listę commitów, które modyfikowały plik szablony.css w folderze css. Można to stosować dla całego folderu.

`git log --oneline --patch -- css` – Wyświetli uproszczoną listę zmian dla folderu „css”.
`git log --oneline --summary -- css` – Wyświetli uproszczoną listę zmian dla folderu „css” (tylko tworzenia i usuwania plików).
`git log --stat --oneline -- css/szablon.css` – Wyświetli uproszczoną listę statystyk zmian dla pliku „szablon.css” z folderu „css”. Działa też na całych folderach.
`git log --format="%h %an %s (%cr)"` – Wyświetlenie listy commitów w formacie: Id Autor Komentarz (Czas).
`git shortlog` – Wyświetla wszystkie commity z podziałem na autorów.

Tag – zakładka, która pozwala łatwiej odnaleźć jakiś commit. Najczęściej oznacza się nimi commity na których zakończono pracę nad daną wersją projektu.
`git tag` – lista utworzonych tagów.
`git show nazwaTagu` – Informacje o commicie na który wskazuje tag.
`git tag nazwaTagu` – Tworzy tag o podanej nazwie na aktualnym commicie.
`git tag nazwaTagu idCommitu` – Tworzy tag o podanej nazwie na commicie o podanym id.
`git tag nazwaTagu idCommitu –a –m "Opis tagu"` – Tworzy tag z informacją o autorze –a i opisem –m na comicie o podanym id.

`git tag –d nazwaTagu` – Usuwa tag o podanej nazwie.
`git push --tags` – Wysyła tylko tagi do zdalnego repozytorium.
`git push alias nazwaTagu` – Wysyła dany tag do zdalnego repozytorium o podanym aliasie.
`git push alias –d nazwaTagu` – Usuwa dany tag ze zdalnego repozytorium o podanym aliasie.
Jak nazywać tagi: https://semver.org/

`git mv nazwa nowaNazwa` – Zmiana nazwy pliku / folderu.

`git mv nazwaPlikuLubFolderu ścieżkaDoNowegoFolderu` – Przeniesienie pliku / folderu.

PRZENOSZENIE PLIKU DO FOLDERU NADRZĘDNEGO:
`cd nowyFolder` – Wejście w folder gdzie jest plik do przeniesienia.
`git mv nazwaPliku ..`– Przeniesienie pliku do folderu nadrzędnego.
`cd ..` – Powrót do folderu nadrzędnego.

USÓWANIE Z MOŻLIWOŚCIĄ PRZYWRÓCENIA:
`git rm –f nazwaPliku` – **Usunięcie z folderu roboczego oraz z repozytorium** pliku który jest śledzony (świeci na zielono lub był zapisany w poprzednim commicie) z możliwością przywrócenia jeśli wcześniej nie został wykonany zapis (commit).
`git rm –r -f nazwaFolderu` – To samo tylko dla usunięcia folderu.
`git rm -r –cached nazwaFolderu` – Usunięcie folderu / pliku tylko z git, nadal pozostaje na dysku w folderze roboczym.

Przywracanie po usunięciu:

`git reset` – Usunięcie zmiany z kolejki oczekiwania (przenosi zmianę z kolejki oczekiwania na zapis z powrotem do katalogu roboczego).
`git restore ścieżka/nazwaPliku` lub `git checkout ścieżka/nazwaPliku` – Przywrócenie (wczytanie) poprzedniej wersji pliku / folderu do katalogu roboczego.

USUWANIE BEZ MOŻLIWOŚCI PRZYWRÓCENIA:
`git clean` – Usuwa z folderu roboczego pliki bez możliwości przywrócenia które nie są śledzone (nie zostały dodane do git lub zostały z niego usunięte – świecą na czerwono).
`git clean -n` – Wyświetla listę plików które można usunąć.
`git clean -nd` – Wyświetla listę plików i folderów które można usunąć.
`git clean -idf` – Usuwanie w trybie interaktywnym (i) plików i katalogów (d) z zawartością (f – daje uprawnienia administratora) wybierz numer 4 „ask each" – wybieramy co chcemy usunąć a co nie wpisując: y – yes lub n – no.

PRZYWRACANIE POPRZEDNICH ZAPISÓW (COMMITÓW):
`git log --oneline` – Lista utworzonych commitów wraz z ich Id.
`git checkout d647ac9 folder/index.js` – Przywrócenie tylko pliku „index.js" znajdującego się w folderze „folder" do wersji z zapisu o Id „d647ac9", aby to zapisać należy stworzyć nowy commit.

`git checkout IdCommitu` – Przejście na zapis (commit) o podanym id (Będą widoczne wszystkie pliki zapisane w tym commicie). Aby z niego wyjść należy użyć np.: `git switch -`. Jeśli wprowadzimy jakieś zmiany w tym commicie nie zostaną one w nim zapisane. Można zapisać je jedynie na nowym branchu: git add => git commit i najważniejsze `git switch -c NazwaNowegoBranchu` – Przechodzi na nowy branch i zapisuje na nim zmiany.

`git checkout nazwaBranchu` lub `git switch -` – Przejście na dowolny branch (- poprzednio użyty).

`git revert IdCommitu` – **ODWRACA ZMIANY** Tworzy nowy commit na podstawie commitu poprzedzającego commit o podanym Id. Inaczej tworzy nowy commit na podstawie commitu o podanym Id ale z cofniętymi zmianami. Automatycznie zostanie włączony terminal Vim do Skomentowania stworzonego commitu („i" – kursor w polu do pisania, wyjście: Esc -> „:" -> „wq" -> Enter.

`git reset IdCommitu` – Przywraca zmiany do wskazanego punktu w historii zmian. To polecenie modyfikuje historię zapisów (commitów) i **nie powinno być używane na commitach które znajdują się już w publicznym repozytorium (np. na Git Hub)** ponieważ można utracić zmiany. Git reset domyślnie wywołuje flagę mixed.
`git reset --mixed IdCommitu` – Zachowuje wszystkie zmiany w historii zapisów do commitu o podanym Id, a wszystko co było później usuwa – działa tylko na historie git, NIE WPŁYWA NA ZAWARTOŚĆ FOLDERU ROBOCZEGO (pliki w git są Untracked, można je dodać poleceniem git add). Po zapisaniu nowego commitu zmiany przejdą na niego, a ten o podanym Id nadal będzie zawierał to co wcześniej.

`git reset --soft IdCommitu` – To samo co mixed ale usuwa tylko commity, zostawia historię plików i automatycznie dodaje ją do aktualnego commitu („automatycznie używa git add ”).
`git reset --hard IdCommitu` – Przywraca wszystko do stanu z comitu o podanym Id (usuwa późniejsze commity, historię plików, pliki w folderze roboczym, a także przywraca te pliki, które znajdowały się w nim wcześniej.

ZAPIS ZMIAN NA STOSIE – w przypadku gdy jesteśmy na innym branchu niż chcieli byśmy zapisać zmiany. Np. zapomnieliśmy przejść z brancha master na branch roboczy.
`git stash` – Zapis zmian na stosie.
`git stash branch nazwaNowegoBrancha` – Utworzenie nowego brancha i przeniesienie na niego zmian. Zmiany zostaną automatycznie usunięte z pamięci stosu.

`git stash push -m "Komentarz"` – Zapis zmian na stosie z możliwością dodania komentarza.
`git stash push –m "Komentarz" –u` – Zapis zmian które nie są śledzone.

`git checkout nazwaBranchu` lub `git switch nazwaBranchu` – Przejście na dowolny branch.

`git stash pop stash@{0}` – Przeniesienie zmian o Id: „stash@{0}” ze stosu na projekt. Bez podania Id przenosi domyślnie najnowsze zmiany. (Zmiany są usuwane z pamięci stosu).
`git stash apply` – To samo co pop, ale zmiany zostają w pamięci stosu.

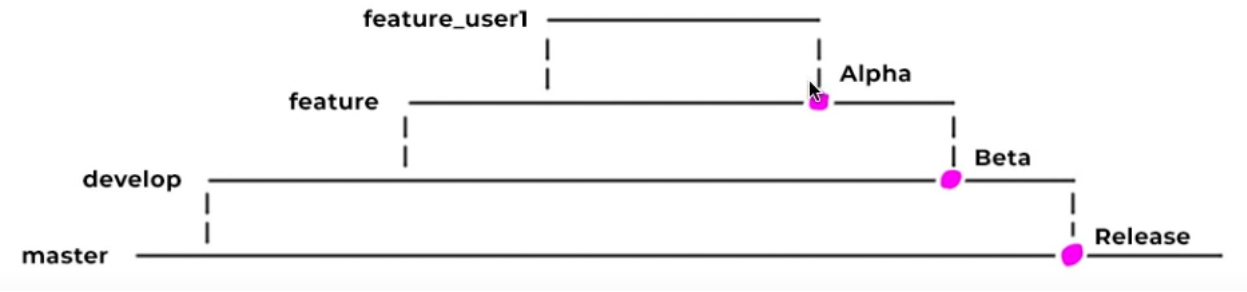
`git stash list` – Pokazuje listę zmian obecnych na stosie z ich Id, oraz nazwą brancha pochodzenia.

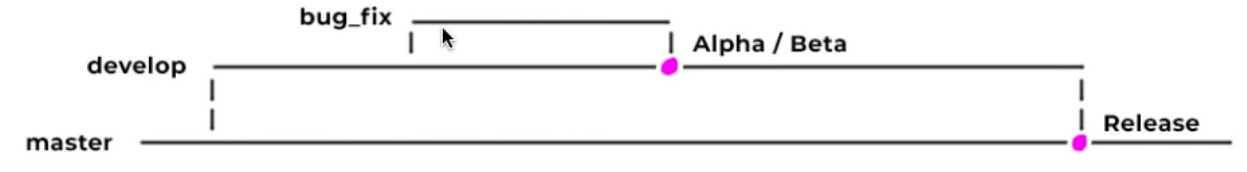
`git stash drop stash@{0}` – Usuwa z pamięci stosu tylko zmianę o id: „stash@{0}”.
`git stash clear` – Czyści całą pamięć stosu.

Tworzenie branchy:

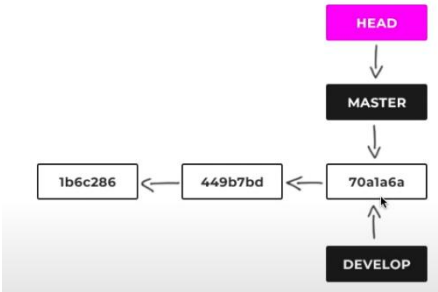
Tworzy się następujące typy branchy:
- Master / Main branch (tworzony automatycznie) – Główna gałąź projektu, powinna zawierać najnowszą stabilną wersje projektu. Możliwość wprowadzania zmian powinna być ograniczona.
- Develop / Dev branch – Gałąź dla wersji testowej projektu (alpha, beta), zwykle połączona z serwerem testowym.
- Feature branch – Gałąź dla pracy nad nowymi funkcjami. Jednej funkcji dedukujemy jeden branch. Po ukończeniu zmiany trafiają na branch Dev.
- User branch – Dla indywidualnej pracy członka zespołu nad zmianami. Ukończone zmiany trafiają na Feature branch.
- Test / Bugfix branch – Dla szybkiego sprawdzenia czegoś lub do wprowadzanie szybkich zmian z powodu występującego błędu.

Schemat tworzenia branchy:

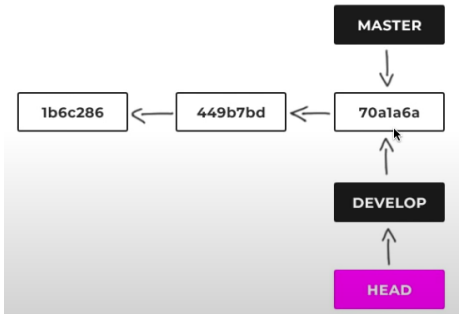




Stworzenie brancha DEVELOP (git branch develop)

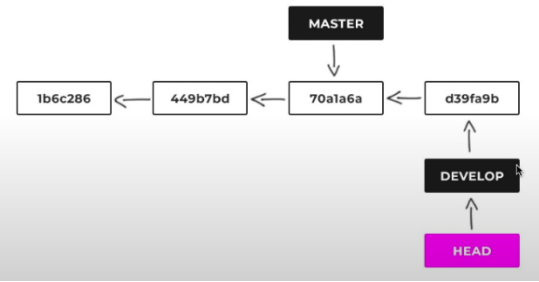


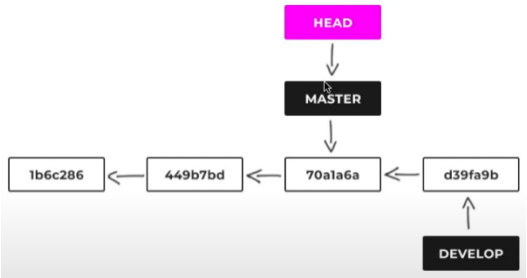
Przejście na branch DEVELOP (git checkout develop)



Stworzenie nowego snapshota na branchu DEVELOP (git commit)

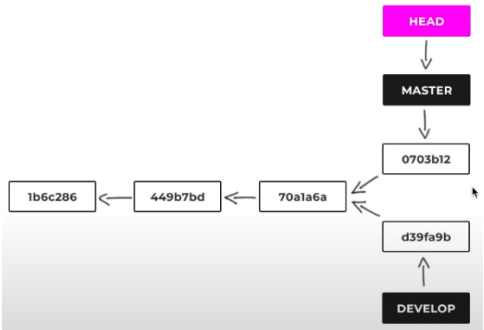
Przejście na branch MASTER (git checkout master)

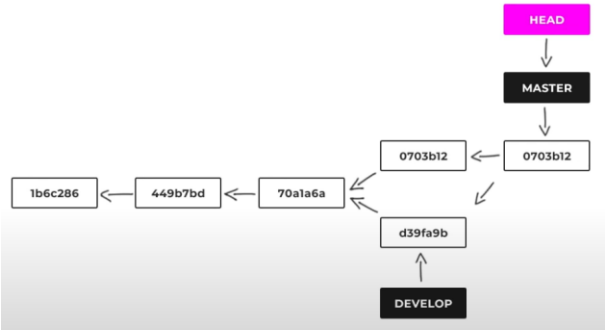




Stworzenie nowego snapshota na branchu MASTER (git commit)

Połączenie brancha DEVELOP z MASTER - przeniesienie danych z DEVELOP na MASTER (git merge develop)





`git branch` – Lista lokalnie utworzonych branchy (Flaga: -a pokaże zarówno lokalne jak i zdalne, -r tylko zdalne).
`git branch nazwaBrancha` – Tworzy nowy branch na bazie brancha w którym aktualnie jesteśmy.
`git branch nazwaBrancha nazwaBranchaBazowego` – Tworzy nowy branch na bazie brancha o podanej nazwie, nie musimy się w nim znajdować.
`git branch nazwaBrancha IdCommitu` – Tworzy nowy branch na bazie commitu o podanym id.
`git branch nazwaBrancha tag` – Tworzy nowy branch na bazie tagu (np.: v2.3).
`git branch nazwaBrancha origin/nazwaZdalnegoBrancha` – Tworzy nowy branch na bazie brancha na repozytorium zdalnym.

`git checkout nazwaBrancha` – Przełącza się na wybrany branch.
`git checkout -b nazwaBrancha` – Tworzy i od razu przełącza się na stworzony branch.

`git rebase nazwaBranchaWzorcowego` – Jeśli na branchu (branchWzorcowy) który posłużył do utworzenia innego brancha (branch na którym aktualnie jesteśmy) powstał nowy commit, to polecenie przenosi początek (miejsce utworzenia) nowego brancha (branch na którym aktualnie jesteśmy) na nowo utworzony commit w branchu wzorcowym. Co upraszcza historię zmian i zwiększa jej czytelność. **Git rebase** modyfikuje historię, używaj go przemyślanie i **nie używaj na zdalnym repozytorium.**

Jeśli wystąpi **konflikt** podczas polecenia **rebase** należy:
- otworzyć plik z konfliktem i go rozwiązać (Edytować zawartość otoczoną liniami: <<<<< ===== >>>>> we właściwy sposób i następnie usunąć te linie).
- `git add nazwaPliku` – dodać edytowany plik do kolejki.
- `git rebase --continue` – Kontynuuje (wykonuje) rebase.

Można też użyć:
- `git rebase --skip` – Pomija commit z konfliktem I kontynuuje rebase.
- `git rebase --abort` – Anuluje polecenie rebase.

`git rebase nazwaBranchaWzorcowego –i` – rebase w trybie interaktywnym, otworzy się terminal VIM. Można w nim:
- Połączyć wiele commitów w jeden **squash** lub **fixup**.
- Zmieniać wiadomość commitów *
- Usuwać commity *

`git merge nazwaBrancha` – Łączy branche. Przenosi zmiany z brancha „nazwaBrancha" na branch na którym aktualnie jesteśmy.

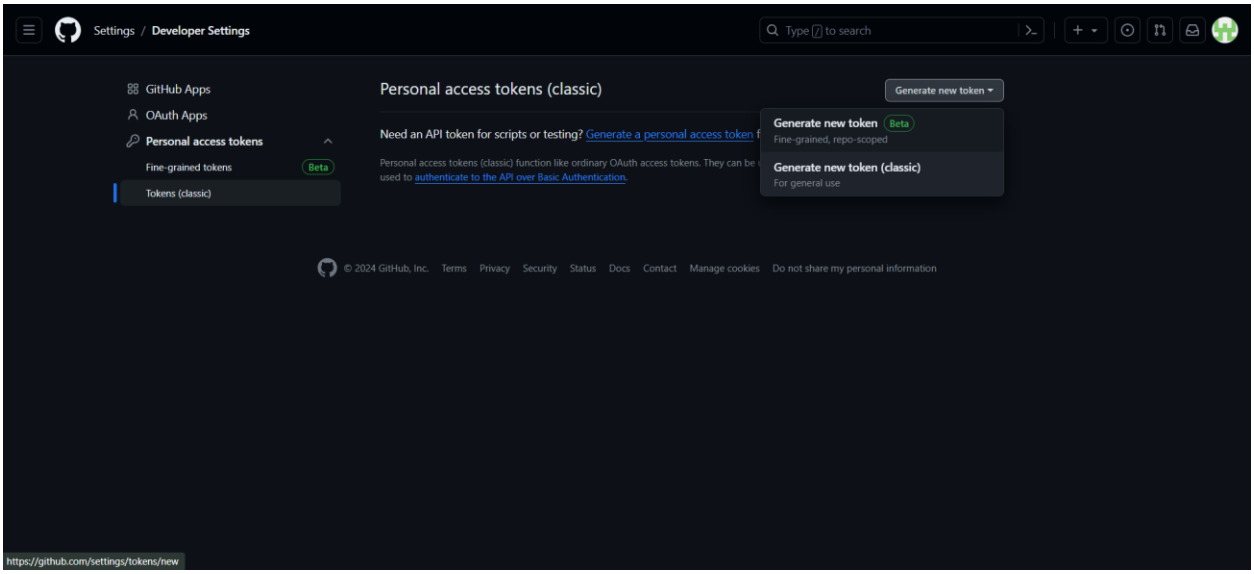
`git log --graph --decorate --all --oneline` – Polecenie przedstawi historie repozytorium (tworzenie branchy, commitów itp.) Należy czytać od dołu do góry.

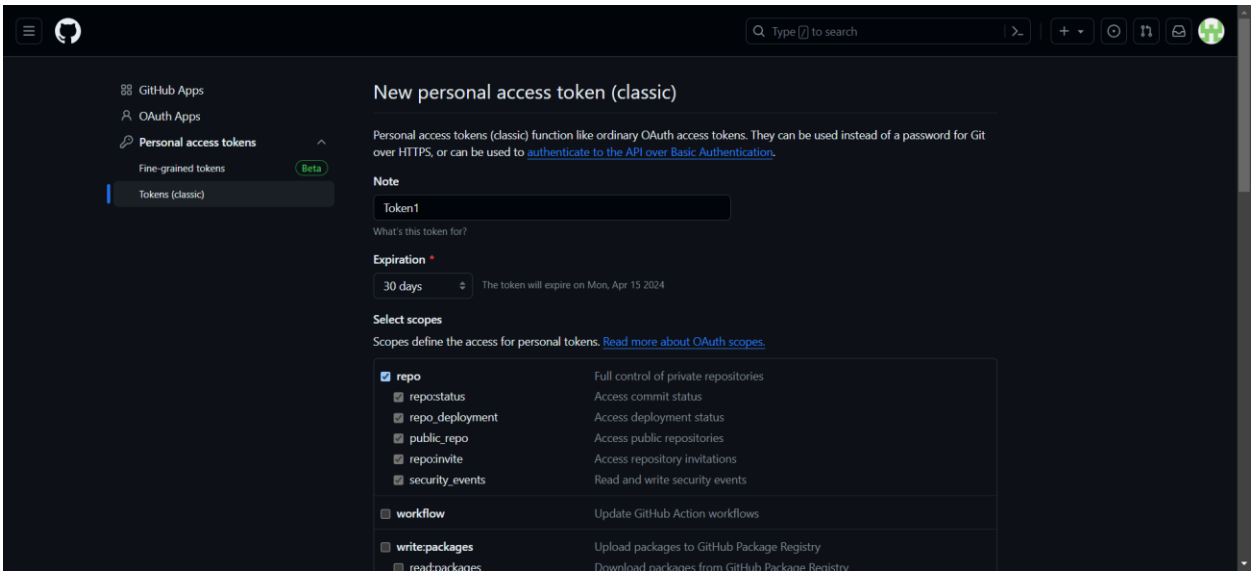
`git branch –d nazwaBrancha` – Usuwa branch.
`git branch -m staraNazwa nowaNazwa` – Zmiana nazwy dowolnego brancha
`git branch -m nowaNazwa` - Zmiana nazwy aktualnie używanego brancha.

TWORZENIE ZDALNEGO REPOZYTORIUM (GitHub)

Metody uwierzytelniania na GitHub – Łączenie git z GitHub:
HTTPS – wykorzystuje login oraz hasło do GitHub. *

Wykorzystanie tokenu: *
Utworzenie tokenu: prawy górny róg / Settings / Developer settings / Personal access tokens / Generate new token (classic)
Skopiuj token ponieważ więcej go nie zobaczysz.





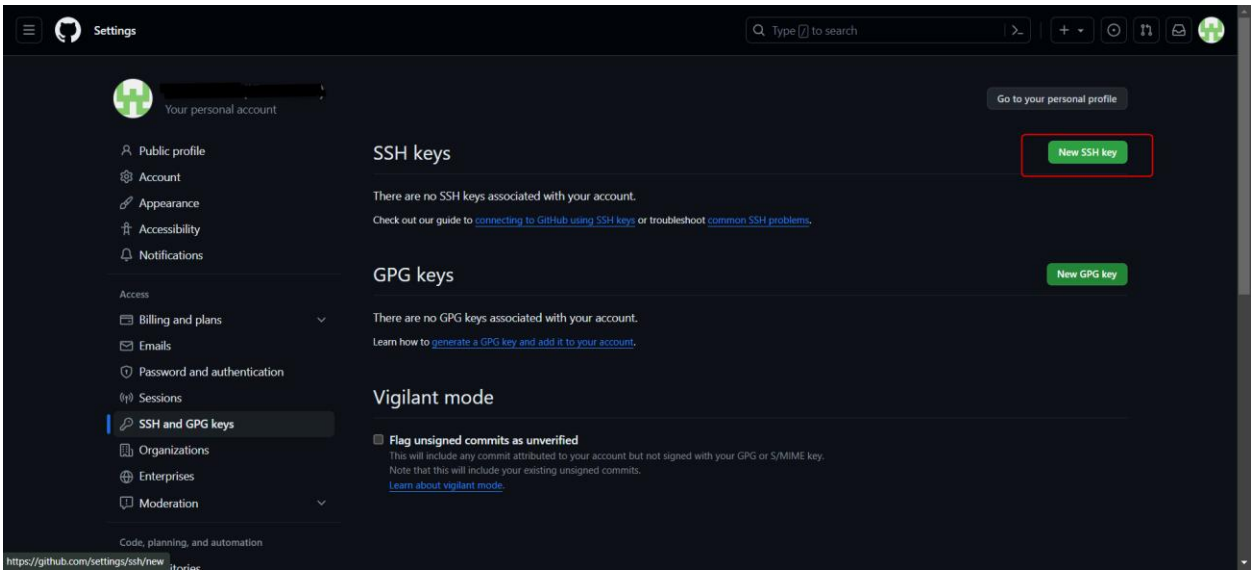
Otwórz Git Bash i w lokalizacji repozytorium wpisz:
git remote set-url origin https://github.com/IP3Dteam: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@github.com/IP3Dteam/Testy.git
* Alias **origin** można zamienić na dowolną inną nazwę.

SSH – wykorzystuje klucz publiczny

Otwórz Git Bash, zostań w domyślnej lokalizacji: ~ lub tam przejdź: cd ~
Wpisz: `ssh-keygen`
- prośba o podanie nazwy pliku z kodem SSH – można zostawić puste i wcisnąć Enter.
- podaj hasło i wciśnij Enter.

- powtórz hasło i wciśnij Enter.

Plik z kluczem publicznym: nazwa.pub lub przy zostawieniu pustego pola: id_123.pub znajduje się przy lokalizacji domyślnej: ~ w: C:\Users\Użytkownik\.ssh
Należy skopiować całą zawartość tego pliku (długi ciąg znaków) i wkleić na GitHub w: prawy górny róg / ikona / settings / SSH and GPG keys / New SSH key.



Podczas dodawania aliasa linku dostępu do repozytorium zdalnego trzeba wykorzystać link SSH.
`Np. git remote add origin git@github.com:NazwaKonta/Testy.git`
Przy każdej operacji na repozytorium zdalnym trzeba podać hasło wpisane wcześniej przy generowaniu kodu SSH.

`git remote add origin https://github.com/JakisUrzytkownik/NazwaRepozytorium.git` - Podłączenie zdalnego repozytorium i określenie aliasu / nazwy (domyślnie origin) przechowującego link do niego. Link należy skopiować z GitHub, panel wybranego repozytorium, zakładka code (HTTPS lub SSH).
`git remote -v` – Lista aliasów repozytoriów zdalnych wraz z linkami.

`git push -u origin master/main` – Pierwsze przesłanie plików na zdalne repozytorium, flaga –u (--set-upstream) umożliwia podanie linku zdalnego repozytorium (origin), oraz nazwy brancha na który wysyłamy zmiany (master – nazwa musi być taka sama jak nazwa brancha lokalnego). Następnie na tym branchu można już korzystać z krótkiej komendy: `git push`.

PRZED ROZPOCZĘCIEM PRACY NALEŻY SPRAWDZIĆ, CZY REPOZYTORIUM LOKALNE JEST AKTUALNE ZE ZDALNYM.

`git clone linkDoRepozytoriumZdalnego` – **Klonuje / Pobiera** do aktualnie otworzonego folderu całe repozytorium z wszystkimi plikami i ich historią z repozytorium zdalnego. Przydatne podczas rozpoczęcia pracy nad istniejącym już projektem.

`git clone --single-branch --branch nazwaBrancha linkDoRepozytoriumZdalnego` – Klonuje tylko wybrany branch.

`git log --all --decorate --oneline --graph` – Sprawdzenie historii zmian.
`git fetch` – Pobranie historii zmian ze zdalnego repozytorium.
`git merge origin/master` – Aktualizacja lokalnego brancha master do zdalnego o aliasie origin. Jeśli na zdalnym branchu master zaszły jakieś zmiany to polecenie zaktualizuje lokalny branch.
`git pull alias nazwaBrancha` – łączy polecenia `git fetch` i `git merge`.

Zmiana nazwy brancha w repozytorium zdalnym:

1. Zmień nazwę branch'a lokalnie → `git branch -m staraNazwa nowaNazwa`.
2. Usuń zdalny branch, którego nazwa została zmieniona → `git push origin --delete staraNazwa`.
3. Stwórz zdalny branch z nową nazwą → `git push -u origin nowaNazwa`.

Frok – stosowane gdy nie mamy dostępu do głównego repozytorium (nie jesteśmy jego właścicielem). Tworzymy kopie wybranego repozytorium na naszym koncie GitHub, na kopie przesyłamy zmiany, a następnie z naszego konta GitHub możemy przesłać zmiany na oryginalne repozytorium (o ile właściciel zaakceptuje Twoje zmiany - Pull request (w przypadku przesłania na branch główny master / main)). Aby stworzyć Fork wejdź na link wybranego repozytorium ze swojego konta GitHub, a następnie kliknij przycisk Fork.

1. Pobierz repozytorium zdalne na lokalne.: `git clone`
2. Utwórz nowy branch i wprowadź zmiany: `git branch nazwaBrancha, git add .`, `git commit`
3. Wyślij zmiany do zdalnego repozytorium: `git push –u origin nazwaBrancha`
4. Utwórz pull request na GitHub (zapytanie właściciela o wgranie Twojej wersji na jego repozytorium)
5. Jeśli pull request został zaakceptowany to na swoim git:
   - `git checkout master` – Przejdź na branch główny
   - `git branch –d nazwaBrancha` – Usuń wcześniej utworzony branch
   - `git push origin –delete nazwaBrancha` – Usuń branch z repozytorium zdalnego (Fork)
   - `git remote add alias linkDoRepozytoriumZdalnego` – Dodaj dodatkowe zdalne repozytorium (orginalne repozytorium nie Fork)
   - `git fetch alias` – Pobierz historie zmian oryginalnego repozytorium

- `git log --all --decorate --oneline --graph` – Sprawdź historie zmian, czy repozytorium lokalne, zdalne orginalne i zdalne Fetch wskazują na najnowszy commit
- `git merge alias/master` – Zaktualizuj lokalne repozytorium na podstawie zdalnego oryginalnego
- `git push` – Zaktualizuj zdalne repozytorium Fork
- `git log --all --decorate --oneline --graph` – Ponownie Sprawdź historie zmian

Na końcu właściciel głównego repozytorium powinien pobrać najnowszą wersje na swoje repozytorium lokalne: `git pull`

Konflikty powstają, podczas łączenia zmian `git merge`, najczęściej gdy edytujemy ten sam fragment pliku na 2 branchach a potem je łączymy.

```
<!-- branch#1 | commit: 26c1583 -->
<html>
  <body>
    <h1>overment.com</h1>
  </body>
</html>
```

```
<!-- branch#2 | commit: 52d1b62 -->
<html>
  <body>
    <h2>overment</h2>
  </body>
</html>
```

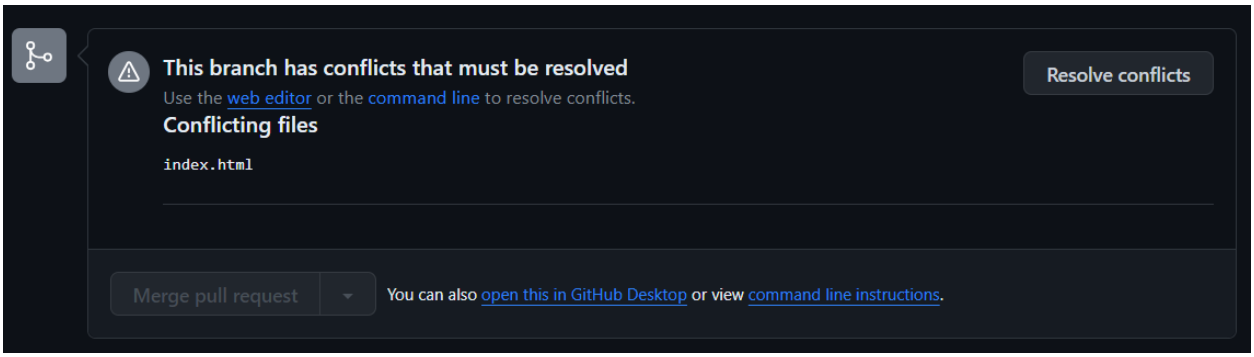
```
<!-- branch#1 -->
<html>
  <body>
<<<<<<< HEAD
      <h1>overment.com</h1>
=======
      <h2>overment</h2>
>>>>>>> branch#2
  </body>
</html>
```

W takim przypadku należy otworzyć plik w którym jest konflikt i go edytować (ustalić która zmiana jest dobra, mogą być obie, należy jedynie usunąć linie wprowadzone przez git). Konflikt znajduje się po między:

<<<<<<<<<<<
Zawartość brancha do którego dołączamy (ważniejszy) np. master
==========
Zawartość brancha dołączanego np. feature
>>>>>>>>>>>

Po właściwym uporządkowaniu kodu należy usunąć znaki dodane przez git: <, =, >, itp, zapisać plik, dodać do kolejki oczekiwania: `git add` i utworzyć commit: `git commit`. Po wszystkim można usunąć branch który dołączaliśmy: `git branch –d nazwaBrancha`.

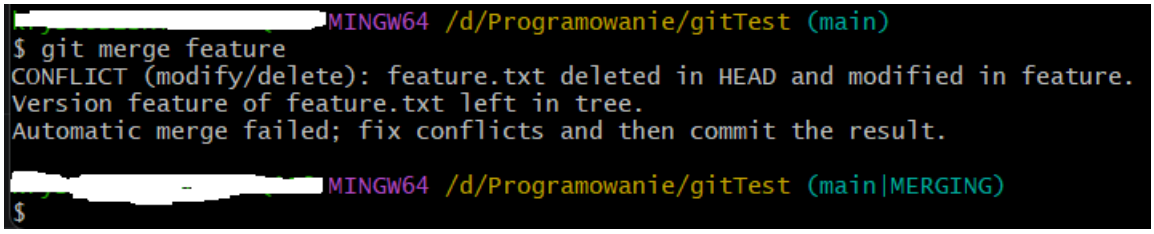
Konflikt na repozytorium zdalnym:



Łącze web editor, oraz przycisk Resolve conflicts – otworzą możliwość edycji w przeglądarce (prostsza metoda).
Łącze command line otworzy podpowiedź z komendami jak to rozwiązać za pomocą Git Bash i edytora na komputerze:

- `git checkout -b nazwaBranchaDoPołączenia branchWzorcowy` – stworzenie na bazie brancha wzorcowego (np. master / main) nowego brancha (tymczasowego) i przeniesienie się na niego.
- `git pull linkDoRepozytorium nazwaBranchaDoPołączenia` – pobranie brancha (np. feature) który później zostanie dołączony do brancha głównego (np. master / main).
- `code . nazwaPliku` - Otworzyć plik w którym jest konflikt i go rozwiązać, zapisać zmiany: `git add .`, `git commit.`
- `git checkout branchWzorcowy` – Przejście na branch główny.
- `git merge --no-ff nazwaBranchaDoPołączenia` – Połączenie brancha tymczasowego z głównym. Flaga `--no-ff` tworzy nowy commit w momencie łączenia branchy.
- `git push alias branchWzorcowy` – Wysłanie zmian na zdalne repozytorium o aliasie (np. origin) na branch (np. master / main - nazwa musi być taka sama jak brancha lokalnego).

Konflikt z usuniętym plikiem – gdy na jednym branchu usuwamy plik a na drugim wprowadzamy do niego zmiany. Podczas łączenia tych branchy: `git merge nazwaBrancha` zostaniemy zapytani czy chcemy usunąć plik czy zachować go ze zmianami.



Poleceniem: `git status` można sprawdzić dostępne opcje.

```
$ git status
On branch main
Your branch is ahead of 'origin/main' by 4 commits.
  (use "git push" to publish your local commits)

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add/rm <file>..." as appropriate to mark resolution)
        deleted by us:   feature.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

`git add nazwaPliku` – Zachowuje plik

`git rm nazwaPliku` – Usuwa plik

`git commit` - Po wszystkim należy zapisać zmiany